

Слайд 1 – общая инфа

Дарова, сегодня поговорим про ci/cd или о том как оформить процесс поддержки вашего приложения, его обновления, правильного размещения на серверах, чтобы уменьшить количество багов, которые появляются из-за проблем инфраструктуры.

Слайд 2 – Докер

Докер – программная платформа для контейнеризации, которая позволяет упаковать приложение со всеми его настройками, библиотеками в единый контейнер, который будет запускаться на всех устройствах одинаково, на которых установлен докер. Аналоги. Podman, containerd

Слайд 3 – создание образа

Образ – это шаблон, который позволяет создавать контейнеры по своему образу.

Чтобы создать образ, нужно создать Dockerfile, в котором будут находиться инструкции. Список инструкций:

- 1) **FROM** – задает базовый образ, от которого создается контейнер с приложением, должна быть первой инструкцией в файле, за исключением ARG
- 2) **WORKDIR** – задает рабочую область внутри контейнера для последующих команд
- 3) **COPY** – копирует файлы или директории из контекста сборки
- 4) **RUN** – выполняет команду во время сборки
- 5) **CMD** – задает команду и аргументы выполнения
- 6) **ENV** – устанавливает переменные окружения, которые будут доступны как во время сборки, так и при работе запущенного контейнера
- 7) **ARG** — определяет переменные сборки (build-time arguments). Они доступны только на этапе сборки, не сохраняются в рантайме
- 8) **EXPOSE** — декларативно указывает, какие сетевые порты приложение слушает внутри контейнера (служит документацией, сам по себе порт на хосте не открывает).

Каждый образ – набор наложенных друг на друга слоев, каждая инструкция, модифицирующая файловую систему создает новый слой.

...

```
FROM mcr.microsoft.com/dotnet/sdk:9.0
```

```
WORKDIR /src
```

```
# Копируем всё сразу
```

```
COPY . .
```

```
# При любом изменении хоть одного файла с кодом
```

```
# шаг скачивания зависимостей будет запускаться заново!
```

```
RUN dotnet restore
```

```
RUN dotnet build -c Release
```

...

...

FROM mcr.microsoft.com/dotnet/sdk:9.0

WORKDIR /src

1. Копируем ТОЛЬКО файлы манифестов зависимостей

COPY MyProject.csproj .

2. Скачиваем библиотеки. Этот слой закэшируется!

Пока вы не добавите новый NuGet-пакет в .csproj, Docker будет брать шаг из кэша мгновенно

RUN dotnet restore

3. И только теперь копируем изменяемый исходный код

COPY . .

4. Быстрая компиляция

RUN dotnet build -c Release --no-restore

...

Слайд 4 – сборка и запуск

Чтобы собрать образ нужно в директории с Dockerfile выполнить `docker build .`

Чтобы посмотреть библиотеку образов на пк нужно выполнить `docker images ls`

чтобы запустить нужно выполнить `docker run -p <host_port>:<container_port> --name <container_name>`

Слайд 5 – докер композ

Dockerfile – это про создание одного контейнера, докер композ – это инструмент для запуска многоконтейнерных приложений. Докеркомпозиционный файл – один yaml файл

services:

Наш бэкенд-сервис

api-service:

build:

context: .

dockerfile: Dockerfile

ports:

- "8080:8080"

environment:

- DB_CONNECTION=Host=postgres-db;Database=appdb;Username=postgres;Password=secret
- REDIS_HOST=cache-service:6379

depends_on:

postgres-db:

condition: service_healthy

networks:

- app-network

База данных

postgres-db:

image: postgres:16-alpine

environment:

POSTGRES_DB: appdb

POSTGRES_PASSWORD: secret

volumes:

- pgdata:/var/lib/postgresql/data

healthcheck:

test: ["CMD-SHELL", "pg_isready -U postgres"]

interval: 5s

timeout: 5s

retries: 5

networks:

- app-network

Кэш

cache-service:

image: redis:7-alpine

networks:

- app-network

volumes:

pgdata: # Постоянное хранилище для БД

networks:

app-network:

driver: bridge

📖 **Внутренний Service Discovery (DNS):** Внутри сети Compose контейнеры находят друг друга просто **по имени сервиса**. Бэкенду не нужно знать IP-адрес базы: достаточно указать хост postgres-db, и встроенный DNS Docker автоматически перенаправит запрос на нужный контейнер.

📖 **Изолированные сети по умолчанию:** Если сеть не объявлена явно, Compose автоматически создаёт дефолтную общую сеть для всех сервисов проекта, изолируя их от внешнего мира и других проектов на машине.

📖 **Управление порядком запуска (depends_on + healthcheck):** Обычный depends_on: [postgres-db] лишь гарантирует, что контейнер с базой стартовал, но не ждёт, пока СУБД инициализирует таблицы и откроет сокет. Использование флага condition: service_healthy в связке с секцией healthcheck позволяет приложению запускаться строго тогда, когда база реально готова принимать подключения.

📖 **Секция volumes:** Декларирует именованные тома, чтобы данные (например, таблицы базы данных) сохранялись на диске между перезапусками стека.

Слайд 6: базовые команды по управлению

Базовые команды по управлению :

- 1) docker compose up -d – запуск приложения в detached режиме, то есть логи не будут писаться вам в терминал
- 2) docker compose ps – отображает статус запущенного проекта
- 3) docker compose stop – останавливает контейнеры без удаления
- 4) docker compose down - останавливает контейнеры и удаляет их и виртуальные сети
- 5) docker compose down -v – останавливает контейнеры, удаляет их, виртуальные сети и тома тоже.

Хоть docker compose – примитивный оркестратор, в разработке в основном используется k8s, docker compose очень хорош при локальной разработке, когда нужно быстро протестировать фичу, без создания новых подов и излишних объявлений.

Слайд 7: кubernetes

Докер композ позволяет запускать приложения на одном конкретном сервере. Есть инструмент, который позволяет управлять сразу кластером серверов, это кubernetes. Kubernetes – оркестратор, который управляет развертыванием контейнеров, его циклом жизни, может сам поддерживать жизнеспособность контейнера.

Главная идея k8s – скрыть железо от разработчика. Мы просто декларативным образом с помощью yaml файла задаем количество реплик контейнера, сколько ему выдавать ядер, оперативки, по какому домену они будут доступны. K8s берет реализацию на себя.

Основные понятия:

- Под – наименьшая и самая базовая единица развертывания
- Service – абстракция которая предоставляет постоянный ip-адрес и днс имя и балансирует нагрузку между подами
- Deployment – управляющий контроллер над подами, который обеспечивает:
 - Декларативное состояние: например replicas: 3
 - Бесшовное обновление: деплоймент не убивает все поды разом, он поочередно убивает поды по-одному (можно настроить), чтобы не было простоя.
 - Rollback – можно откатиться если что-то пошло не так
- ConfigMap – служит для хранения несекретных данных.

apiVersion: v1

kind: ConfigMap

metadata:

name: app-config

data:

ASPNETCORE_ENVIRONMENT: "Production"

ENABLE_ANALYTICS: "true"

- Secret – служит для хранения чувствительной информацией. Под капотом данные в манифесте хранятся в base64 кодировке
- Helm – пакетный менеджер для k8s. Шаблонизатор. Chart – шаблоны манифестов. Values.yaml – файл с конкретными переменными, которые подставляются в шаблоны. Release – запущенный экземпляр чарта в кластере

Слайд 8: Архитектура k8s:

1) control-plane

- a. kube-apiserver – точка входа для запросов kubectl
- b. etcd – key-value хранилище, где лежит состояние кластера и конфигурации
- c. kube-scheduler – распределяет контейнеры по серверам в зависимости от свободных ресурсов
- d. kube-controller-manager – собственно и выполняет запросы от kube-apiserver

2) worker-node

- a. kubelet – обеспечивает работу подов, включая их контейнеры.
- b. kube-proxy – задает правила сети для поддержки services

оффтоп:

Kubectl – официальная утилита, консольная

Openlens- подключается к файлу .kube/config визуализирует нагрузку по процессору, памяти, отображает дерево ресурсов, позволяет редактировать yaml прямо в окне программы.

Слайд 9: Основы ci\cd

Ci\cd методология автоматизации сборки, тестирования и доставки приложения. Позволяет уменьшить downtime при релизе и понизить порог входа, уменьшает мануальные действия.

Continuous Integration (Непрерывная интеграция): практика регулярного слияния кода всех разработчиков в общую ветку (по несколько раз в день). Каждый коммит автоматически собирается и тестируется. **Цель:** как можно раньше обнаружить баги и конфликты слияния, пока они стоят дешево.

Continuous Delivery (Непрерывная доставка): автоматическая подготовка проверенного и собранного артефакта (например, Docker-образа), готового к раскатке на продакшен в любой момент. Сам деплой на боевой сервер запускается человеком по нажатию кнопки («ручной аппрув»).

Continuous Deployment (Непрерывное развертывание): полная автоматизация без участия человека. Если код прошел все проверки и тесты в ветке main, он автоматически выкатывается прямо на продакшен-серверы.

Слайд 10: основные понятия ci\cd

- 1) пайплайн – сценарий автоматизации. Единый скрипт в yaml формате, который платформа берет на исполнение
- 2) stages – логические разделы конвейера, выполняются строго последовательно, если test stage упал, тогда deploy даже не начнет выполняться
- 3) jobs – единицы работы внутри одного стэйджа, могут выполняться параллельно.

Слайд 11: основные stages жизненного цикла.

Stages – этапы доставки, развертывания и т.д.

- 1) build – проверка компиляции приложения, его компонентов.
- 2) Test – автоматический прогон unit-тестов. Интеграционных тестов с реальной базой, сканеров уязвимости
- 3) Release – отправка образов в хранилище
- 4) Deploy – развертывание новой версии на тестовый стенд или на продакшен.

Инструменты: gitlab ci\cd – прописываем пайплайн в .gitlab-ci.yml в корне репы.

Слайд 12: Триггеры запуска пайплайнов

Пайплайн запускается при каком-то событии:

- 1) Push в ветку
- 2) Pr, Mr
- 3) Tags
- 4) По крон-расписанию
- 5) Прокликать кнопки в ui интерфейсе

Слайд 13: IaC (infrastructure as code)

IaC – подход к управлению и настройке IT-инфраструктуры через конфиг файлы, а не ручным способом через скрипты и тому подобное. Описываемый желаемый результат, а не задаем алгоритмы.

Проблемы до IaC:

- 1) Каждый сервер хранил собственное состояние кода, потому что из-за человеческого фактора сисадмины могли задеплоить разные версии
- 2) Новое развертывание версии занимало много времени и требовало сложных алгоритмов.
- 3) Дрейф конфигурации. Конфиги прода и теста могли отличаться, из-за чего ответ на одинаковые действия могли давать непредсказуемый результат.

Преимущества IaC:

- 1) Версионирование в Git: Любое изменение инфраструктуры проходит через Pull/Merge Request, код-ревью и аудит
- 2) **Идемпотентность (Idempotence):** Фундаментальный принцип IaC. Сколько бы раз вы ни запускали скрипт — один раз или сто раз подряд — результат будет одинаковым
- 3) **Неизменяемая инфраструктура (Immutable Infrastructure):** Вместо того чтобы накатывать обновления на работающий сервер, старый сервер уничтожается, а на его место поднимается новый из свежего образа с уже примененными изменениями.